Walchand College of Engineering, Sangli
Department of Computer Science and Engineering

Practical No. 6

Study and implementation of node.js

Name Atharv Vinayak Kulkarni

Prn - 245109074

Batch - S5

Problem Statement 1: Introduction to Node.js

---

1. What is Node.js? Difference from Apache/PHP?

Answer:

- Node.js is a **JavaScript runtime environment** built on Chrome's V8 engine.
- It allows running JavaScript **outside browser (server-side)**.

Apache/PHP:

- Apache is a **web server**
- PHP is a **server-side language**

Key Difference:

- Node.js uses **event-driven, non-blocking I/O**
- Apache/PHP use **blocking, thread-based model**

Meaning:

- Node.js handles **many users with single thread**
- PHP creates **new thread per request**

---

2. Purpose of V8 Engine

Answer:

- V8 engine converts **JavaScript → machine code**
- Written in C++
- Makes execution **very fast**

That's why Node.js is fast compared to traditional systems

---

3. Single-threaded Event-driven Architecture

Answer:

- Node.js runs on **single thread**
- Uses **event loop** to manage tasks

Flow:

Request → Event Loop → Callback Queue → Execution

It doesn't create multiple threads
Instead, it handles multiple requests using **callbacks/events**

---

4. Why Node.js is Non-blocking?

Answer:

- Node.js does not wait for tasks (like file read, DB call)
- It continues execution

Example:

```
fs.readFile('file.txt', ()=>{});
console.log("Next line executes immediately");
```

File loads in background → program continues

---

5. What is npm?

Answer:

- npm = **Node Package Manager**
- Used to install/manage libraries

Uses:

- Install packages
- Manage dependencies
- Run scripts

```
npm install express
```

---

6. What is Module?

Answer:

- A module is a **separate file containing reusable code**

Export:

```
module.exports = add;
```

Import:

```
const add = require('./file');
```

Helps in **code organization**

---

7. require() vs import

require()	import
CommonJS	ES Modules
Default in Node	Modern JS
Synchronous	Asynchronous

---

8. Custom Module

Example:

```
// math.js
function add(a,b){
return a+b;
}
module.exports = add;
```

Then use:

```
const add = require('./math');
```

---

9. package.json

Answer:

- It is the **configuration file of Node project**

Contains:

- Project name
- Dependencies
- Scripts

Created using:

```
npm init -y
```

---

10. Install packages

Local:

```
npm install express
```

Used only in project

Global:

```
npm install -g nodemon
```

Used anywhere in system

---

11. Async vs Sync

Async	Sync
Non-blocking	Blocking
Faster	Slower
Uses callback	Sequential

Node prefers **async**

---

12. Create HTTP Server

```
const http = require('http');
http.createServer((req,res)=>{
res.write("Hello");
res.end();
}).listen(3000);
```

Basic server without Express

---

13. http vs Express

http	Express
Low-level	High-level
Manual work	Easy
Complex	Simple

Express = shortcut over http

---

14. Handle GET & POST

```
if(req.method === "GET"){
res.end("GET request");
}
if(req.method === "POST"){
res.end("POST request");
}
```

In Express → easier

Problem Statement 2: Middleware

---

1. What is Middleware?

Answer:

- Function that runs **between request and response**

Used for:

- Logging
- Authentication
- Validation

---

2. Custom Middleware

```
const logger = (req,res,next)=>{
console.log(req.url);
next();
};
app.use(logger);
```

`next()` is compulsory

---

3. Execution Order

Answer:

- Middleware runs in **sequence**
- Order matters

Example:

```
app.use(A);
app.use(B);
```

A runs first, then B

Problem Statement 3: File System (fs)

---

1. Read & Write File

```
fs.readFile('file.txt','utf8',(err,data)=>{
console.log(data);
});

fs.writeFile('file.txt','Hello',()=>{});
```

---

2. readFile vs readFileSync

- readFile → async (non-blocking)
- readFileSync → sync (blocking)

---

3. Check File Exists

```
fs.existsSync('file.txt');
```

---

4. Async File Handling

- Uses callbacks/promises
- Does not block execution

Problem Statement 4: Database

---

1. Connect MySQL

```
const mysql = require('mysql2');
const db = mysql.createConnection({
host:'localhost',
user:'root',
password:'',
database:'test'
});
```

---

2. mysql2

- Library to connect Node + MySQL
- Supports promises

---

3. CRUD

Create:

```
INSERT INTO users VALUES(1,'Atharv');
```

Read:

```
SELECT * FROM users;
```

Update:

```
UPDATE users SET name='A';
```

Delete:

```
DELETE FROM users;
```

Problem Statement 5: REST API

API Routes

POST /books
GET /books
PUT /books/:id
DELETE /books/:id

Features

- Add book
- Update book
- Delete book
- Filter data

Authentication

- Use middleware
- JWT / session

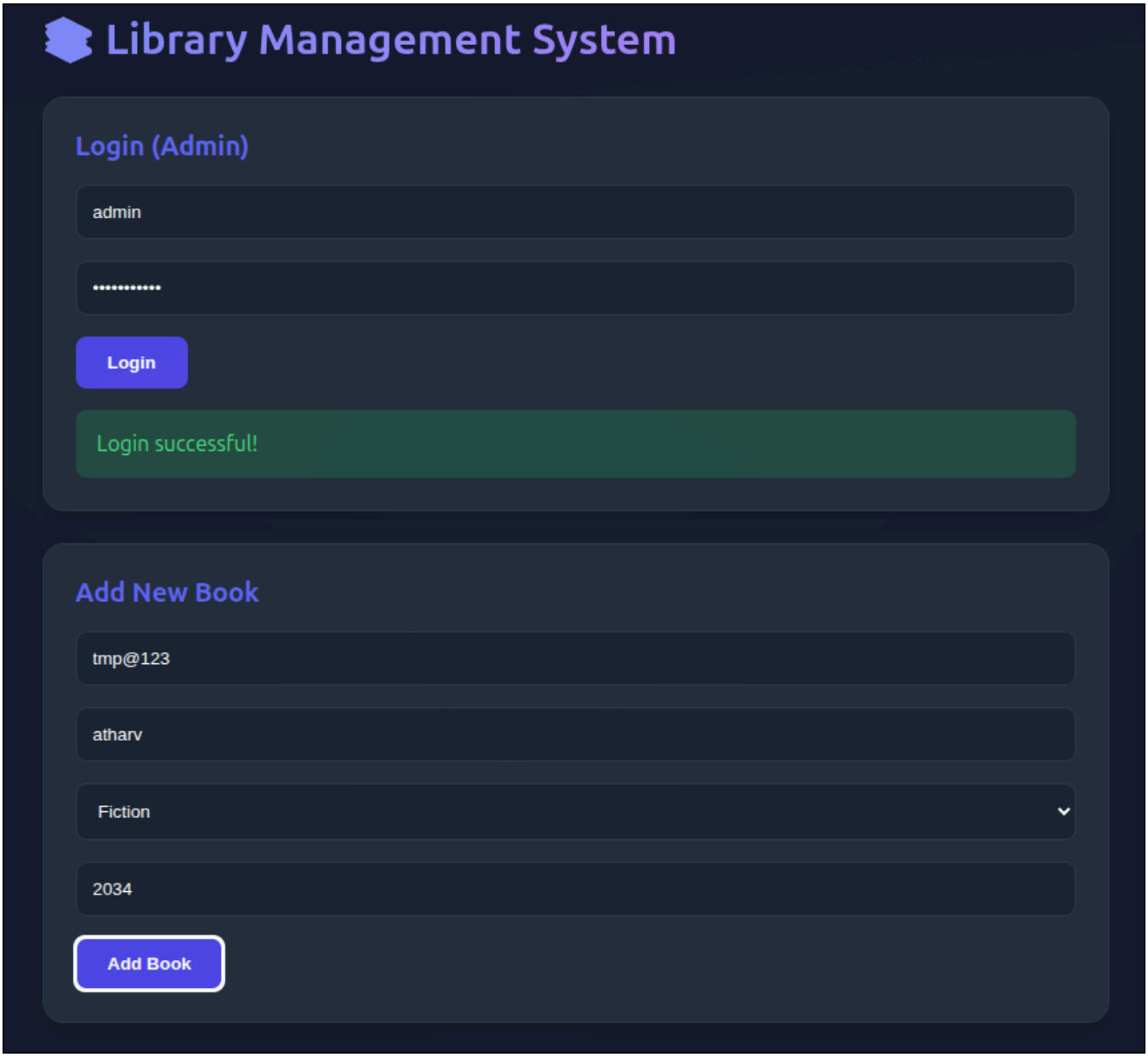
File Upload (Multer)

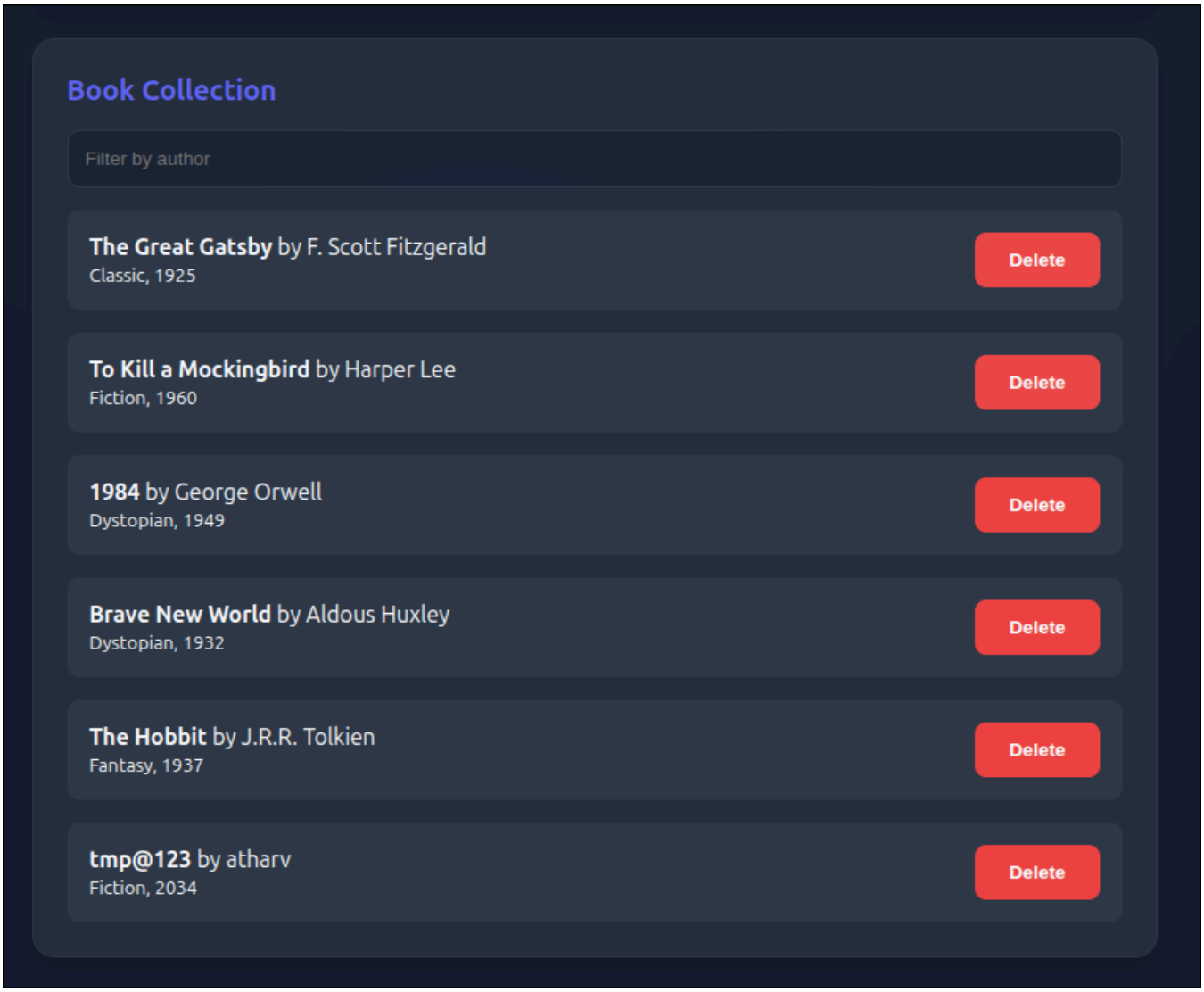
```
const multer = require('multer');
const upload = multer({ dest:'uploads/' });

app.post('/upload', upload.single('file'), (req,res)=>{
res.send("Uploaded");
});
```

Features

- Upload
- View
- Download/Delete
- Auth protected




Book Collection
Filter by author
The Great Gatsby by F. Scott Fitzgerald
Classic, 1925
Delete
To Kill a Mockingbird by Harper Lee
Fiction, 1960
Delete
1984 by George Orwell
Dystopian, 1949
Delete
Brave New World by Aldous Huxley
Dystopian, 1932
Delete
The Hobbit by J.R.R. Tolkien
Fantasy, 1937
Delete
tmp@123 by atharv
Fiction, 2034
Delete